

YARA: Writing Detection Rules

Why YARA

YARA is a pattern-matching language for files and memory. Malware hunters, EDR vendors, and threat intel teams all use it. A rule = what to look for + when to fire.

Anatomy of a rule

```
rule Suspicious_PowerShell_Cradle
{
  meta:
    author = "zero-day-student"
    description = "detects common download-cradle strings"
    score = 40
  strings:
    $s1 = "DownloadString" ascii nocase
    $s2 = "IEX(" ascii nocase
    $s3 = "WebClient" ascii nocase
    $s4 = { 81 EC 00 00 00 00 } // potential decode opcode
  condition:
    uint16(0) == 0x5a4d and // MZ header
    ( filesize < 500KB and 2 of them )
}
```

The string toolbox

ascii / wide / nocase / fullword modifiers

Hex patterns { 6A 00 6A 00 }

Regex via /reg exp/

Multiple conditions: all of them , 2 of (\$a*) , @s1[1]

Anti-FP craft

Require a 'family marker' + a 'behavior marker' together

Scope by size (e.g., only under 1MB)

Reference stable metadata (MZ header, section names, unique mutex)
rather than single common string

Test against a corpus of clean files before shipping

Where rules live

Local: YARA CLI on your triage

Feeds: **yara-rules** projects (YARA-Hunter, Neolex), vendor packs

EDR: most modern EDRs accept Sigmas for detections and YARA for
static/memory matches

A professional loop

1. Analyze sample -> pick 2 distinctive static markers
2. Write rule -> test on sample (must fire) and clean corpus (must not)
3. Ship it to your repo; iterate on real-world feedback

A YARA rule is a hypothesis about what a family MUST contain. When it
false-positives, the hypothesis was weak (bad marker), not the tool.

[← Sandboxing & Malware Analysi...](#)

Module complete ✓